Complemento — 24/10/2026

Este bloco entra na frente de cada PDF do pacote. O corpo do documento continua abaixo. O panorama para apresentar o Conecta+ a uma igreja nova está em **Apresentação para o cliente**.

Quiz semanal

- Caminho da equipe: engrenagem, Culto e Eventos, Quiz semanal.
- Antes de transcrever, a tela mostra tema, pregador, data da celebração e duração do vídeo.
- A transcrição avança em trechos. O botão de busca vira uma barra de progresso.
- São geradas 15 perguntas. A publicação exige exatamente 10. O membro só vê o quiz publicado e responde essas 10, com 10 pontos cada.
- A transcrição da prévia pode ser baixada em PDF.
- Refazer as 15 perguntas usa o texto já gravado. Uma pregação que já virou quiz não se repete.
- O ranking no perfil só aparece para quem concluiu a tentativa.

Pequeno caixa

- Caminho de quem gerencia: engrenagem, Finanças e Inteligência, Pequenos Caixas.
- Caminho de quem controla: Perfil, Pequeno caixa, ou o atalho do Financeiro. É preciso ser o responsável do caixa ativo e ter a permissão de controle.
- Cada caixa tem valor-base, dia de fechamento e responsável. O saldo só muda quando a conferência aprova.
- Despesa e reposição exigem comprovante. A reposição é o valor que falta para o valor-base e só entra depois das despesas pendentes.
- Fim de semana e feriado empurram a prestação para o próximo dia útil.
- Excluir uma despesa já enviada é um pedido, concluído no painel amarelo. Excluir o caixa apaga lançamentos e comprovantes.
- O passo a passo dos dois lados está no procedimento do Pequeno caixa.

Aviso de novos membros

Quem já entrou na régua de acolhimento, ou cuja recepção familiar já foi processada, deixa de contar como pendência da etiqueta amarela do Início.

Controle de acesso — modelo e inventário

Este documento rastreia **como o app-igreja identifica usuários hoje**, quais **telas**, **tabelas** e **campos** existem no ecossistema, e como a tabela de relacionamento proposta se encaixa.

Script SQL: `scripts/access-control-schema.sql`

Manual operacional (dia a dia): `MANUAL_CONTROLE_ACESSO.md`

Pacote: `PACOTE_3_GOVERNANCA_TI.md` · **Índice:** `INDICE_DOCUMENTACAO.md` · **Camadas de segurança:** `CAMADAS_SEGURANCA.md`

Status da implementação (atualizado em 05/10/2026)

Documento de encerramento da sessão: o que já está pronto, o que falta e qual é o **próximo passo** recomendado.

Concluído no Supabase (Passos 1–7)

Item	Status
Tabelas `access_resources`, `access_roles`, `access_grants`, `profile_access_roles`	Criadas
Correção de índices únicos (`role_id` / `profile_id` parciais)	Aplicada (evita erro 23505 com `profile_id` null)
Funções `profile_has_access` e `profile_has_access_by_phone`	Criadas
Seed de recursos, papéis e grants (`member`, `super_admin`, `events_admin`)	Executado
Perfil administrador (`04b919ba-38b4-4fe5-a371-2e98e9acbc0d`) com `super_admin + member`	Configurado
Todos os perfis em `profiles` com papel `member`	Configurado
Teste SQL: `profile_has_access` → manutenção `true` para super_admin	OK

Concluído no app (Passo 9)

Item	Arquivo(s)
Sessão: `user_phone + user_profile_id` após login/cadastro	`lib/userSession.ts`, `app/index.tsx`, `app/register.tsx`
Cliente ACL: RPC `profile_has_access` / `sessionHasAccess`	`lib/accessControl.ts`
Engrenagem do dashboard só para quem tem `view` em `/maintenance-dashboard`	`app/(tabs)/dashboard.tsx`
Bloqueio da tela `maintenance-dashboard` sem permissão	`app/maintenance-dashboard.tsx`

Logout limpa sessão (telefone + profile_id)	`app/(tabs)/dashboard.tsx`
Passo 9b: carrossel só com cards permitidos (`dashboard.card.*`)	`lib/accessControl.ts`, `app/(tabs)/dashboard.tsx`
Passo 9c: colunas visíveis/editáveis em Dados cadastrais	`lib/accessControl.ts`, `app/manage-profile.tsx`
Passo 9d: `update_profile_field` e `update_profile_access_pin` validam `can_update`	`scripts/access-control-profile-write-rpc.sql`
Passo 9e: UI admin de papéis e grants (somente `super_admin`)	`components/MaintenanceAccessControlCard.tsx`, `scripts/access-control-admin-rpc.sql`
Passo 9f: RLS nas tabelas com `profile_has_access` + header `x-profile-id`	`scripts/access-control-table-rls.sql`, `lib/supabaseSessionFetch.ts`

Entregas recentes (jun/2026)

Item	Detalhe
Relatórios de Despesas (RD)	Telas `/expense-report` e hub no `/financial`; WhatsApp ao tesoureiro na submissão; tesouraria na manutenção (`expense-reports-*.sql`)
Financeiro membro	Saldo bancário por conta; hub com RD destacado
Financeiro manutenção	Carga/esvaziar lote com versão REALIZADO/PLANEJADO; seções colapsáveis; accordion (uma seção aberta)
Recepção Familiar	Formulário `/cadastro-familia/` → fila na manutenção (`recepcao-cadastro-familiar.sql`)
Mudança de Papéis	Card pastoral/super_admin (`access-control-pastoral-role-change.sql`)
Cadastro de Usuário	`maintenance.card.profile_cadastro` — busca, CEP, exclusão completa
Acessos de Usuários	`maintenance.card.profile_access_insights` — histórico de logins e telas visitadas *(super_admin)*
Gestor em Controle de Acesso	Opera papéis e grants sem listar, visualizar ou editar Super Administrador, seus acessos ou PIN/senha
Modo Ghost	ACL, perfil, telefone, família e dados usam a identidade efetiva do alvo; cobrança e instância permanecem na identidade real
Sessão assinada	`profile_sessions` + header `x-session-token` (prioridade sobre `x-profile-id`)

Telemetria de uso	Tabelas `profile_app_access_events` e `profile_app_access_screen_visits`; RPCs admin e `record_profile_app_access_screen_visit`
Mapa ACL PDF	`npm run build:access-roles-pdf` → `PAPEIS_CONTROLE_ACESSO.pdf`
Índice do aplicativo	`/(tabs)/index` — atalhos com etiquetas para todos os cards
Manutenção — card menu	Primeiro card do carrossel com etiquetas dos módulos
Marca d'água	Global via `AppShell`, exceto login; alinhada ao frame do card
Performance navegação	Cache em memória de ACL, perfil de sessão e audiência familiar; sem refetch completo a cada foco de tela
Mapa — detalhe de pin	Recurso `/mapa-geolocalizacao/detalhe-pin` — mapa geral para membros; detalhe alheio só pastoral/super_admin (`access-control-map-pin-detail.sql`)
Audiência familiar	União de fontes inclui congregados; dependentes com `accepted ≠ false` (`family-event-audience-members.sql`)
ACL — visão por recurso	Aba Papéis: toque no marcador colorido do recurso → papéis como linhas com Ver/Editar; botão Voltar
Rodapé do dashboard	Botão Menu expandido; engrenagem alinhada à direita do card (`CarouselFooterNav`)

Ainda não feito (próximas sessões)

- Papel `events_admin` atribuído a pessoas da equipe de eventos (só seed SQL hoje).
- RLS em tabelas auxiliares (`checkins` , `escalas_*` , ...).
- Views com mascaramento de colunas sensíveis em `profiles` .

Outras entregas da mesma época (fora do ACL)

- Senha de acesso em Dados cadastrais (PIN, seção recolhível, olho, validação).
- Gerenciar Família: seção "Adicionar membro" recolhível; busca por nome; **transferência** entre famílias com confirmação; **herança de endereço completo** ao aceitar, transferir ou adicionar membro (`lib/inheritFamilyAddress.ts` , RPC `accept_managed_member_into_family`).
- Dashboard: carrossel com `‹` / `›` , indicador `1 / N` , badge do card ativo; card **Dízimos e Ofertas** sempre visível; card **SALA(S)** filtrado por família do usuário.
- UX compartilhada: `lib/uiTokens.ts` , ícones coloridos no Menu e na Manutenção, chips segmentados no Coração Aberto.

Próximo passo recomendado (quando retomar)

Roadmap ACL (fases 9a–9f) **concluído**. Guards de rota nas telas principais via `hooks/useScreenAccessGuard.ts`.

Próximas melhorias opcionais:

1. Enforcement ACL em RPCs de manutenção (escalas, financeiro em lote, etc.).
2. RLS em tabelas auxiliares (`checkins`, `escalas_log`, `tipos_escala`, ...).
3. Views com mascaramento de colunas sensíveis (`access_pin`, `cpf`) em `profiles`.

Supabase (obrigatório após 9f): execute `scripts/access-control-table-rls.sql` no SQL Editor.

Teste rápido (9f):

```
-- Simula header do app (substitua pelo seu profile_id)
select set_config('request.headers', '{"x-profile-id":"<uuid>"}', true);
select public.session_has_resource_access('table', 'profiles', 'view');
-- esperado: true para member com grant
```

Lembrete ao testar no celular: após mudar papéis no Supabase, use **Sair** e entre de novo para gravar `user_profile_id` na sessão.

---

1. Situação no banco vs. no app (referência)

Aspecto	Hoje
Identidade no app	`profiles.id` (login por PIN em `profiles.access_pin` + telefone em `AsyncStorage`)
Supabase Auth	Opcional (`profiles.auth_user_id`); muitos usuários **não** têm linha em `auth.users`
Autorização	RLS com `profile_has_access` nas tabelas principais (9f) + RPCs `security definer` para fluxos sem sessão
Manutenção	Engrenagem e rota protegidas via `profile_has_access` (Passo 9 parcial)
Cards do dashboard	Filtrados por `profile_has_access` (Passo 9b)
Demais telas	Guards de rota em manutenção, perfil, família, pastoral, financeiro e LGPD
Campos sensíveis	Ocultos no cliente; RPCs `update_profile_field` / PIN validam `can_update` por coluna (9d)

O banco **já responde** permissões via `profile_has_access`; o app consulta para **manutenção** e **cards do dashboard**.

---

2. Sujeito da permissão

Use sempre **`profiles.id`** como "usuário" do ACL.

- Não use só `auth.users.id` — pedidos pastorais e RPCs já documentam o desvio (`pastoral-requests-fields.sql`).
- O app pode resolver `profile_id` a partir do telefone da sessão (`find_profile_id_by_phone` / SELECT em `profiles`).

2.1 Identidade efetiva no Modo Ghost

- Telas, listas e permissões resolvem o alvo com `loadEffectiveSessionProfile`, `resolveEffectiveProfileId` e `getEffectiveUserPhone`.
- O bypass de `super_admin` do operador fica desligado para a ACL simulada.
- O auditor entra e permanece na rota escolhida; grant negado do alvo não provoca retorno ao Início nem bloqueio "Sem acesso nesta simulação".
- Iniciar/encerrar Ghost são as únicas transições automáticas para o Início. Billing, paywall e seleção de instância continuam vinculados ao operador real.

---

3. Inventário de telas (`resource_type = 'screen'`)

resource_key	Rota / origem	Observação
`screen:/`	`app/index.tsx`	Login
`screen:/register`	`app/register.tsx`	Cadastro
`screen:/dashboard`	`app/(tabs)/dashboard.tsx`	Painel principal
`screen:/maintenance-dashboard`	`app/maintenance-dashboard.tsx`	Manutenção (eventos, monitor salas)
`screen:/manage-profile`	`app/manage-profile.tsx`	Dados cadastrais
`screen:/manage-members`	`app/manage-members.tsx`	Gerenciar família
`screen:/pastoral`	`app/pastoral.tsx`	Coração Aberto (formulário)
`screen:/pastoral-history`	`app/pastoral-history.tsx`	Meus pedidos
`screen:/financial`	`app/financial.tsx`	Relatórios financeiros (leitura)
`screen:/expense-report`	`app/expense-report.tsx`	Relatório de Despesas (RD)

`screen:/lgpd`	`app/lgpd.tsx`	Termos LGPD
`screen:/lista-familias`	`app/lista-familias.tsx`	Diretório de famílias
`screen:/visitantes-cadastro-rapido`	`app/visitantes-cadastro-rapido.tsx`	Cadastro rápido e QR do visitante
`screen:/documentos-oficiais`	`app/documentos-oficiais.tsx`	Documentos publicados da igreja
`screen:/apoio-mutuo`	`app/apoio-mutuo.tsx`	Serviços da comunidade
`screen:/atribuicoes`	`app/atribuicoes.tsx`	Papéis operacionais; pastoral e super_admin

Cards do dashboard (`screen:dashboard.card.*`)

resource_key	Card	content
`screen:dashboard.card.event_alt`	Agenda da Família	`event_alt`
`screen:dashboard.card.qr`	Check In	`qr`
`screen:dashboard.card.kids_teens`	SALA(S)	`kids_teens`
`screen:dashboard.card.offerings`	Dízimos e Ofertas	`offerings`
`screen:dashboard.card.pastoral`	Coração Aberto	`pastoral`
`screen:dashboard.card.members_list`	Lista de membros	`members_list`
`screen:dashboard.card.birthdays`	Aniversariantes	`birthdays`
`screen:dashboard.card.vigilance_scales`	Escalas	`vigilance_scales`
`screen:dashboard.card.parking_vehicle_v2`	Estacionamento	`parking_vehicle_v2`
`screen:dashboard.card.grouped_manage`	Menu (perfil + família)	`grouped_manage`

Visibilidade condicional de cards (parâmetros/evento) continua no app; o ACL define se o usuário **pode** ver o card quando ele estaria disponível.

As rotas dedicadas e o menu/"Eu quero…" são a navegação publicada. O antigo carrossel do Painel não deve ser usado como inventário do produto.

Recursos administrativos restritos

- `maintenance.card.profile_access_insights` (**Acessos de Usuários**) é exclusivo de `super_admin`, inclusive nas RPCs de listar e limpar histórico.
- O papel `gestor_controle_acesso` pode administrar o ACL, mas o escudo autoritativo SQL (`assert_gestor_super_admin_shield` e filtros de visibilidade) impede

qualquer acesso ao perfil/papel `super_admin`, aos registros de acesso desse perfil e às colunas de PIN/senha.
- O cliente aplica os mesmos filtros em `lib/gestorControleAcessoSecurity.ts` como defesa em profundidade.

---

4. Inventário de tabelas (`resource_type = 'table'`)

Tabelas usadas pelo app (Supabase `public`):

resource_key	Uso principal
`table:profiles`	Login, perfil, LGPD, endereço, PIN
`table:members`	Família, check-in, listas
`table:events`	Agenda, manutenção, salas
`table:event_registrations`	Check-in / salas Kids-Teens
`table:profile_vehicles`	Veículos no perfil e estacionamento
`table:pastoral_requests`	Pedidos pastorais
`table:pastoral_reason_categories`	Motivos (leitura)
`table:pastoral_reason_subcategories`	Submotivos (leitura)
`table:app_parameters`	Parâmetros globais (PIX, QR, prefixo família, **`LGPD_Ativo`**) — escrita por **super_admin** via RPC `salvar_app_parameter_admin`
`table:families`	Dados de família (`useFamilyData`)
`table:vigilancia_*`	Escalas (import/histórico — conferir nomes no Supabase)

Atualizar a lista após `information_schema.tables` no projeto se houver tabelas só no banco.

O texto e a ativação da LGPD são por instância/tenant. A tela deve carregar o conteúdo da igreja ativa; não se reutiliza texto de outra instância, mesmo quando o mesmo telefone existe em mais de uma igreja.

---

5. Inventário de campos (`resource_type = 'column'`)

Formato: `column:<tabela>.<coluna>`.

`profiles` (cadastro + sensíveis)

Colunas conhecidas no app (`manage-profile.tsx` , `register.tsx`):

Coluna	Sensível	Notas
`full_name`, `phone`, `birth_date`, `email`	médio	Contato / identificação
`cpf`	alto	Oculto na UI padrão
`access_pin`	crítico	Só RPC `update_profile_access_pin`; exige `can_update` (9d)
`address_*`, `cep`	médio	Endereço
`lgpd_*`, `medical_food_alerts`	alto	Privacidade / saúde
`family_id`, `codigo_membro`, `role`	médio	Escopo familiar / papel
`selfie_url`	médio	Imagem
`auth_user_id`, `id`, `created_at`, `updated_at`	sistema	Bloqueados em `update_profile_field`

`members`

Coluna	Notas
`full_name`, `phone`, `birth_date`, `relationship`, `family_id`	Gerenciar família
`accepted`	Reconhecimento na família

`events`

Campos editáveis em `maintenance-dashboard` / `maintenanceEventForm` : `name` , `event_date` , `event_local` , `max_capacity` , `kids_room` , `teens_room` , `parm_ofertas` , `is_locked` , `is_visible` , etc.

`pastoral_requests`

`request_for` , `beneficiary_*` , `destination_label` , `profile_id` , `message` , status, etc. (`pastoral-requests-fields.sql`).

Regra sugerida: `can_view` na tabela não implica todas as colunas — conceda colunas sensíveis (`cpf` , `access_pin`) só a papéis administrativos.

---

6. Modelo proposto (3 tabelas + função)

```
erDiagram
  profiles ||--o{ profile_access_roles : tem
  access_roles ||--o{ profile_access_roles : agrupa
  access_roles ||--o{ access_grants : recebe
  profiles ||--o{ access_grants : direto
  access_resources ||--o{ access_grants : alvo

  access_resources {
    uuid id PK
    text resource_type
    text resource_key
    text label
  }
  access_roles {
    uuid id PK
    text code UK
    text name
  }
  access_grants {
    uuid id PK
    uuid role_id FK
    uuid profile_id FK
    uuid resource_id FK
    boolean can_view
    boolean can_update
  }
```

`access_resources` (catálogo)

Define **o que** pode ser protegido: tela, tabela ou coluna.

`access_roles` (papéis)

Ex.: `member`, `family_acceptor`, `pastoral`, `events_admin`, `super_admin`.

`access_grants` (relacionamento pedido)

Uma linha = permissão de um **papel** *ou* de um **perfil** sobre um recurso:

- `can_view` — visualizar (tela, listagem, SELECT de campo)
- `can_update` — alterar (formulário, UPDATE, RPC de escrita)

Exatamente um de `role_id` ou `profile_id` deve estar preenchido.

`profile_access_roles`

N:N entre `profiles` e `access_roles`.

Função `profile_has_access(profile_id, resource_type, resource_key, action)`

- `action`: `'view'` ou `'update'`
- Suporta curinga `*` no final da chave (ex.: `table:profiles` não inclui colunas; `column:profiles.*` todas as colunas)
- **Modo legado:** se não existir nenhum grant no sistema, retorna `true` (app continua funcionando até você configurar papéis)

---

7. Papéis sugeridos (seed)

code	Quem	View típico	Update típico
`visitantes`	Sem perfil na sessão ou perfil sem papéis	Login, cadastro, LGPD, check-in QR, eventos públicos	Cadastro e inscrição em eventos
`congregado`	Participante cadastrado	Dashboard básico, perfil, pastoral; sem família/financeiro	Perfil e pedido pastoral
`member`	Membro comum	Próprio perfil (campos básicos), cards dashboard, pastoral próprio	Perfil próprio (campos permitidos), pedido pastoral
`family_acceptor`	Quem aceita familiares	`manage-members`, membros da família	`members` da própria `family_id`
`lider`	Líder de tipo(s) de escala	Painéis de servos/programação + card Escalas	Tipos vinculados em `profile_scale_leadership`
`events_admin`	Equipe de eventos	`maintenance-dashboard`, `events`	CRUD `events`
`tesoureiro`	Tesouraria	Card financeiro, manutenção financeira, RD (AAMM), eventos retroativos	Conciliação RD, importação CSV
`pastoral`	Equipe pastoral	Pedidos (futuro painel)	Triagem `pastoral_requests`
`super_admin`	TI / pastor responsável	*	*

Ordem no painel admin: mesma sequência da tabela acima.

Ajuste conforme a política da igreja.

---

8. Roadmap do app (ordem sugerida)

Fase	Descrição	Status
9a	Manutenção (engrenagem + tela)	Feito
9b	Cards do dashboard (`dashboard.card.*`)	Feito
9c	Campos do perfil (`column:profiles.*`)	Feito
9d	RPCs de escrita	Feito
9e	UI admin de grants	Feito
9f	RLS nas tabelas	Feito

---

9. Checklist operacional

Supabase

- [x] Executar `scripts/access-control-schema.sql` (e correção de índices se necessário)
- [x] `super_admin` no perfil administrador
- [x] `member` em todos os perfis
- [x] Testar `profile_has_access` no SQL Editor
- [ ] Atribuir `events_admin` a quem cuida de eventos (quando definir a equipe)
- [ ] Revisar grants do papel `member` conforme política da igreja

App

- [x] `lib/userSession.ts` + `lib/accessControl.ts`
- [x] Login/cadastro persistem `user_profile_id`
- [x] Dashboard: engrenagem condicional
- [x] `maintenance-dashboard` : guard de acesso
- [x] Dashboard: filtrar cards por ACL (Passo 9b)
- [ ] Supabase: `access-control-member-dashboard-grants.sql` se seed antigo omitiu cards do `member`
- [x] `manage-profile` : colunas visíveis/editáveis por permissão (Passo 9c)
- [ ] Supabase: `access-control-member-profile-columns.sql` se seed antigo omitiu `cpf` / `medical_food_alerts` no `member`

- [x] RPCs: validar `can_update` antes de gravar (Passo 9d)
- [ ] Supabase: `access-control-profile-write-rpc.sql` após deploy do app 9d
- [x] Manutenção: card Controle de Acesso para `super_admin` (Passo 9e)
- [ ] Supabase: `access-control-admin-rpc.sql` após deploy do app 9e
- [x] App: header `x-profile-id` em todas as requisições Supabase (Passo 9f)
- [x] RLS: policies ACL em tabelas principais (Passo 9f)
- [ ] Supabase: `access-control-table-rls.sql` após deploy do app 9f

Arquivos de referência no código

Arquivo	Uso
`lib/accessControl.ts`	Constantes `ACCESS_SCREEN`, `ACCESS_DASHBOARD_CARD`, helpers RPC
`lib/userSession.ts`	`user_profile_id` no AsyncStorage
`lib/supabaseSessionFetch.ts`	Header `x-profile-id` para RLS (9f)
`hooks/useScreenAccessGuard.ts`	Guard de rota por `screen:*`
`scripts/access-control-table-rls.sql`	Policies RLS por tabela
`DASHBOARD_CARDS.md`	Lista de cards e `content`
`MANUTENCAO_ECOSISTEMA.md`	Rotina do módulo de manutenção
`MANUAL_CONTROLE_ACESSO.md`	Manual operacional (papéis, grants, testes, troubleshooting)